Operating Systems

Lab 08

Process Synchronization and Critical Section

Section 1: Race Condition

Example Problem

Two threads increment a shared variable 100000 times each. Observe incorrect results.

Code (C using pthreads)

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void* increment(void* arg) {
    for(int i = 0; i < 100000; i++) {
        counter++;
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter: %d\n", counter);
    return 0;
}
```

Line-by-Line Explanation

- `#include <stdio.h>`: Provides input/output functions like printf
- `#include <pthread.h>`: Enables thread creation and management
- `int counter = 0;`: Shared variable accessed by multiple threads (critical section data)

- `void* increment(void* arg)`: Thread function executed by each thread
- `for(int i = 0; i < 100000; i++)`: Loop to increment counter many times
- `counter++;`: Non-atomic operation (read → modify → write), causes race condition
- `pthread_t t1, t2;`: Thread identifiers
- `pthread_create(...)`: Creates threads that run concurrently
- `pthread_join(...)`: Waits for threads to finish
- `printf(...)`: Displays final value

Concept Insight

The increment operation is not atomic. Multiple threads interleave execution leading to lost updates.

Task 1

Modify the program to run 4 threads instead of 2.

Hint

Create additional pthread_t variables and call pthread_create more times.

Task 2

Print the counter value inside the loop occasionally.

Hint

Use a condition like `if(i % 20000 == 0)` to print intermediate values.

Section 2: Mutex Lock

Example Problem

Fix race condition using mutex.

Code

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;
```

```
void* increment(void* arg) {
    for(int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter: %d\n", counter);
    pthread_mutex_destroy(&lock);
    return 0;
}
```

Line-by-Line Explanation

- `pthread_mutex_t lock;`: Declares a mutex variable
- `pthread_mutex_init(&lock, NULL);`: Initializes the mutex before use
- `pthread_mutex_lock(&lock);`: Acquires the lock before entering critical section
- `counter++;`: Now protected, only one thread executes at a time
- `pthread_mutex_unlock(&lock);`: Releases lock so other threads can enter
- `pthread_mutex_destroy(&lock);`: Cleans up resources after use

Concept Insight

Mutex ensures mutual exclusion by allowing only one thread to execute critical section code at a time.

Task 1

Add a delay inside the critical section.

Hint

Use `sleep(1)` after acquiring the lock.

Task 2

Move the lock outside the loop and observe behavior.

Hint

Place lock before loop and unlock after loop to see reduced concurrency.

Section 3: Binary Semaphore

Example Problem

Use semaphore instead of mutex.

Code

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

int counter = 0;
sem_t sem;

void* increment(void* arg) {
    for(int i = 0; i < 100000; i++) {
        sem_wait(&sem);
        counter++;
        sem_post(&sem);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    sem_init(&sem, 0, 1);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter: %d\n", counter);
    sem_destroy(&sem);
    return 0;
}
```

Line-by-Line Explanation

- `sem_t sem;`: Declares semaphore variable
- `sem_init(&sem, 0, 1);`: Initializes semaphore with value 1 (binary semaphore)
- `sem_wait(&sem);`: Decrements semaphore, blocks if value < 0

- `counter++;`: Critical section
- `sem_post(&sem);`: Increments semaphore, wakes waiting thread
- `sem_destroy(&sem);`: Cleans up semaphore

Concept Insight

Binary semaphore behaves like mutex but without strict ownership rules.

Task 1

Change semaphore value to 2.

Hint

Observe how race condition reappears.

Task 2

Add print statements before and after sem_wait.

Hint

Track thread entry and exit behavior.

Section 4: Counting Semaphore

Example Problem

Allow only 3 threads to access a resource.

Code

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t sem;

void* access_resource(void* arg) {
    sem_wait(&sem);
    printf("Thread %ld entered\n", (long)arg);
    sleep(1);
    printf("Thread %ld exiting\n", (long)arg);
```

```
    sem_post(&sem);
    return NULL;
}

int main() {
    pthread_t threads[5];
    sem_init(&sem, 0, 3);

    for(long i = 0; i < 5; i++) {
        pthread_create(&threads[i], NULL, access_resource, (void*)i);
    }

    for(int i = 0; i < 5; i++) {
        pthread_join(threads[i], NULL);
    }

    sem_destroy(&sem);
    return 0;
}
```

Line-by-Line Explanation

- `sem_init(&sem, 0, 3);`: Allows 3 threads simultaneously
- `sem_wait(&sem);`: Decrements count, blocks if no resource
- `printf("entered")`: Indicates access granted
- `sleep(1);`: Simulates work in critical section
- `sem_post(&sem);`: Releases resource
- Thread array handles multiple threads

Concept Insight

Counting semaphore controls access to multiple identical resources.

Task 1

Increase number of threads to 10.

Hint

Observe grouping of thread execution.

Task 2

Change semaphore value to 1.

Hint

Compare behavior with mutex.

Section 5: Producer Consumer Problem

Example Problem

Implement bounded buffer using semaphores.

Code

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5
int buffer[N];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    for(int i = 0; i < 10; i++) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % N;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void* consumer(void* arg) {
    for(int i = 0; i < 10; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        out = (out + 1) % N;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        printf("Consumed %d\n", item);
    }
    return NULL;
}
```

Line-by-Line Explanation

- `#define N 5`: Buffer size
- `buffer[N]`: Shared circular buffer
- `in, out`: Index pointers
- `empty`: Counts empty slots
- `full`: Counts filled slots
- `mutex`: Ensures mutual exclusion
- `sem_wait(&empty)`: Wait if buffer full
- `sem_wait(&full)`: Wait if buffer empty
- `pthread_mutex_lock`: Protect buffer access
- `sem_post(...)`: Signals availability

Concept Insight

Multiple semaphores handle different conditions: mutual exclusion and resource counting.

Task 1

Add multiple producers.

Hint

Create multiple producer threads.

Task 2

Add print statements showing buffer state.

Hint

Print buffer contents after each operation.

Section 6: Peterson's Algorithm

Example Problem

Implement Peterson for two threads.

Code

```
#include <stdio.h>
#include <pthread.h>
```

```
int flag[2] = {0, 0};
int turn;

void* process0(void* arg) {
    flag[0] = 1;
    turn = 1;
    while(flag[1] && turn == 1);

    printf("Process 0 in critical section\n");

    flag[0] = 0;
    return NULL;
}

void* process1(void* arg) {
    flag[1] = 1;
    turn = 0;
    while(flag[0] && turn == 0);

    printf("Process 1 in critical section\n");

    flag[1] = 0;
    return NULL;
}
```

Line-by-Line Explanation

- `flag[2]`: Indicates interest of each process
- `turn`: Decides priority
- `flag[i] = 1`: Process wants to enter
- `turn = j`: Gives other process priority
- `while(flag[j] && turn == j)`: Wait condition
- Critical section executes when condition false
- `flag[i] = 0`: Leaves critical section

Concept Insight

Peterson ensures mutual exclusion, progress, and bounded waiting using only shared variables.

Task 1

Add loops to repeatedly enter critical section.

Hint

Wrap code in a loop.

Task 2

Add delays to observe alternation.

Hint

Use sleep to visualize process switching.